



DSA Assignment - 4

Unit-4 Assignment: Data Management Mini Toolkit (DMMT)

Course Details

Name of the School: School of Engineering & Technology

Student Name: Tanuj

Section: A

Program (Class / Semester / Batch): B.Tech CSE (AI MI) / 2 /
2025-29

Course Title: Data Structures

Course Code: ETCCDS202

Unit Title: Non-Linear & Advanced Data Structures

Faculty: Neetu Chauhan

Submission Date: 17/04/2026

Task 1 – Binary Search Tree (BST)

In this assignment I implemented a **BST** class with insert, search, delete, and inorder traversal functions in Python. I tested it using the required sequence: insert keys [50, 30, 70, 20, 40, 60, 80], search for 20 and 90, then delete a leaf node (20), a one-child node (60 after inserting 65), and a two-children node (30), printing the inorder traversal after each deletion.

Why inorder traversal prints a BST in sorted order.

Inorder traversal of a BST visits the left subtree, then the current node, then the right subtree. Because all keys in the left subtree are smaller and all keys in the right subtree are larger than the node, this order naturally prints the keys in sorted increasing order.

Time complexity discussion in simple terms (average $O(\log n)$, worst $O(n)$ if skewed).

For time complexity, in the average case when the tree is reasonably balanced, the height is about $\log n$, so insert, search, and delete take about $O(\log n)$ time. In the worst case (highly skewed tree like a linked list), the height becomes n and these operations degrade to $O(n)$.

Task 2 – Graph, BFS and DFS

For the graph module I represented a directed, weighted graph using an adjacency list in Python, with nodes A, B, C, D, E, F and edges: $A \rightarrow B(2)$, $A \rightarrow C(4)$, $B \rightarrow D(7)$, $B \rightarrow E(3)$, $C \rightarrow E(1)$, $D \rightarrow F(5)$, $E \rightarrow D(2)$, $E \rightarrow F(6)$, $C \rightarrow F(8)$. I printed the adjacency list and implemented both BFS and DFS starting from node A, printing the traversal order for each.

Difference between BFS and DFS in simple words.

BFS (Breadth-First Search) uses a queue and explores the graph level by level from the starting node, visiting all neighbors at the current distance before going deeper. DFS (Depth-First Search) uses recursion or a stack and explores as deep as possible along one path before backtracking to explore other branches.

One real-world use case each (e.g., BFS for shortest path in unweighted graph, DFS for cycle detection).

A practical use case of BFS is finding the shortest path in an unweighted graph, for example shortest number of hops between people in a social network. DFS is useful for tasks like cycle detection, topological sorting, and exploring all reachable states or paths in a maze.

Task 3 – Hash Table with Separate Chaining

For the hash table module I implemented a simple integer-key hash table using separate chaining. The hash function is $index = key \% table\ size$, and each index stores a list of (key, value) pairs to form a chain. I used table size 5 and inserted keys 10, 15, 20, 7, 12 to force collisions, then retrieved some keys and deleted one key from a collided bucket, printing the table before and after deletion.

What is a collision and why it happens.

A collision occurs when two or more different keys map to the same table index because the hash function returns the same index for them.

Why separate chaining helps.

Separate chaining handles this by keeping a small linked list or Python list of pairs at that index; new colliding keys are added into the same bucket and search only walks through that short chain.

Expected time complexity: average $O(1)$, worst $O(n)$ if many collisions.

If the load factor is controlled and chains stay short, the expected average time complexity of insert, get, and delete is $O(1)$ (constant time). In the worst case, if many keys collide into a single bucket, the chain can become long and operations can degrade to $O(n)$.

Observations and Learning

By completing this mini toolkit, I connected the theory of non-linear and advanced data structures with practical Python implementations. The BST showed how ordered storage supports efficient search and sorted

traversal, the graph module illustrated how BFS and DFS explore networks, and the hash table demonstrated how hashing and collision handling enable fast key–value access similar to real hash maps and caches.